TurboHap: supplementary methods and benchmark observations

Supplementary File 1 — 22 October 2026.

Companion to *TurboHap: accelerating FindHap haplotype phasing and genotype imputation while preserving output compatibility*, by Adrian R. Carr, Green Neuron AI, Edinburgh, UK.

This supplement gives the benchmark protocol, implementation details, individual observations and dataset-generation recipe. Supplementary File 2 contains the corresponding data, reference sources, reproduction tools and figures. TurboHap v1.0.0 is publicly available at https://github.com/adrian-greenneuron/TurboHap.

S1. Benchmark protocol

The seven workloads combine 10,000 or 100,000 genotyped animals with 1,000, 10,000 or 100,000 markers, plus one million animals with 10,000 markers. Each has four additional ungenotyped pedigree animals. The simulation settings and validation steps are given in Section S5. Each dataset was simulated separately; the two timing repeats use the same input files.

Measurements used ten OpenMP threads on an NVIDIA DGX Spark with 20 logical CPU cores, Ubuntu 24.04, ARM64 and GCC/gfortran 13.3.0. The GPU was unused. Compiler flags were:

```
Fortran: -O3 -fopenmp -fno-underscoring
C++:     -O3 -march=native -funroll-loops -flto
         -std=gnu++17 -fopenmp
```

C++ linking also used `-flto-partition=1to1`. The reported speedups include both implementation and compiler-setting effects.

For each dataset, runs proceeded sequentially as Fortran, TurboHap, LM, LM, TurboHap, Fortran. The environment was `LC_ALL=C`, `OMP_DYNAMIC=FALSE` and `OMP_NUM_THREADS=10`, with affinity permitting CPUs 0–19. There was no exclusive CPU reservation, warm-up run or forced cache eviction. The server also ran other tasks; their load was not systematically recorded. All 42 observations reported zero swaps.

Elapsed wall time includes input, inference, all result writes and LM temporary-file I/O on internal NVMe storage. Comparisons and output hashing follow timing. Peak process RSS was recorded with GNU `time` in KiB and divided by 1,048,576 to obtain GiB. RSS does not capture temporary-disk capacity or all filesystem-cache memory. The CSV and JSON also retain CPU time, page faults and filesystem activity counters.

Runtime and RSS are summarised separately. With two observations, the median is their midpoint. Speedup is the ratio of Fortran to C++ median runtime; memory change is $100 \times$ (C++ median RSS / Fortran median RSS $- 1$). Figure bars show the two-run ranges, not confidence intervals.

S2. Reference software and output comparison

The C++ source is TurboHap v1.0.0. Its source files and CMake configuration match those used to build the measured executable. Source and executable SHA-256 hashes, compiler versions and flags are recorded in `data/turbohap_build_provenance_221026.json` in Supplementary File 2.

The archive also supplies the exact Fortran reference sources, including the gfortran/ARM64 adaptations, in `reference/findhapV3/`. Their hashes match the build metadata. With gfortran 13.3.0 on Linux ARM64, build from that directory using:

```
gfortran findhap.f90 nhash.f90 quicksort.f90 -o findhap \
  -O3 -fopenmp -fno-underscoring
```

The original FindHap code is public domain and retains its USDA attribution. The reference-source README describes its provenance. Compiler and runtime changes may produce a different executable hash.

Every enabled result file is compared byte for byte except `cross.overs`. Its header must match exactly; complete subsequent records are sorted in the C locale, retaining all fields, whitespace and duplicates. TurboHap repeats and modes must match strictly, including crossover order. Console progress and timing messages are excluded.

All 28 C++ runs passed comparison with the first Fortran run for their dataset. The second Fortran run also passed, and all four C++ runs per dataset matched one another exactly. File hashes and comparison outcomes are in `data/turbohap_output_validation_221026.json`. The public example checker separately runs the original example in both modes at one and four threads against saved Fortran hashes.

The included `tools/benchmark_dataset.py` provides an entry point for independent reruns. It uses the same program order, locale, thread setting, timing fields and output-comparison policy. From the extracted archive root, run:

```
python3 tools/benchmark_dataset.py \
  --dataset /path/to/dataset \
  --baseline /path/to/fortran/findhap \
  --turbohap /path/to/TurboHap/build/findhap \
  --work-dir /path/to/new-run-directory
```

This Linux utility requires Python 3.9+, GNU `time` and GNU `sort`. It accepts the four benchmark inputs without priors, runs each program twice with ten threads by default, and stops on a failed run or output mismatch. It retains all outputs, logs and new measurements, so allow space for six complete output sets. These reruns are separate from the supplied study observations.

S3. Implementation details

Storage

TurboHap separates its input cache from mutable working genotypes so that compression does not change how edits propagate between passes. Input records are cached at their declared chip lengths, using two bits for values 0, 1, 2 and missing (5), with byte storage for rows containing other parsed values. Working genotypes occupy a separate packed layout whose segment starts are byte-aligned, preventing workers on adjacent segments from modifying the same byte. Overlapping or nonpartitioned prior layouts instead use dense storage so that aliases of a shared locus see the same edits.

At a segment-length transition, each previous haplotype library retains all slots through its largest referenced identifier. Retaining only the active count would lose identifiers where gaps occur. Assignments and progeny haplotype IDs use 16-bit storage when `maxhap` is at most 65,535, and 32-bit storage above it. Frequencies, counts and animal indices retain their original arithmetic widths. The compiled Fortran reference uses 32-bit default integers for the corresponding assignment and progeny-ID arrays. Thus the six datasets with `maxhap=50000` halve those payloads in C++, whereas `maxhap=500000` on the million-animal case selects 32-bit storage and removes that particular saving. Selection depends on configured capacity, not on the active library size measured at completion. This explains an allocation difference, not the full observed RSS difference, which also includes the genotype stores, indexes, library arenas and allocation lifetimes.

For G genotype records and segment lengths L_s, packed working-genotype storage is $G\sum_s \lceil L_s/4 \rceil$ bytes, in addition to the raw cache and metadata. Assignment and inheritance arrays include terms proportional to pedigree size times segment count. The current haplotype arena depends on configured capacity and segment dimensions. These describe individual allocations; peak RSS also depends on which allocations coexist.

LM stores its immutable raw cache in an immediately unlinked temporary file on the working filesystem. For partitioned layouts, a second file stores mutable working genotypes. Workers load consecutive animal blocks, save edits before replacing them, and keep the segment library and candidate state across blocks. Its genotype-payload budgets are 32 MiB for the raw-cache batch, 64 MiB for input staging and 128 MiB across population-worker buffers. Oversized records and dense prior fallbacks can exceed these buffer targets. Haplotype libraries, assignments, metadata and thread scratch remain resident.

Search and parallel execution

The sampled signature records known alleles at up to 64 evenly spaced positions. Unknown alleles never reject a candidate. The ordered bitmap index is available for population searches with zero allowed conflicts, at least four informative sampled positions and a library of at least 1,024 haplotypes. Unsupported sizes or search conditions use the compact scan. A range-minimum tree preserves the lowest frequency and earliest tied position in the skipped prefix. Signatures and indexes are updated after relevant library changes.

A full scan of H_s candidates with segment length L_s costs $O(H_s \times L_s)$ in the worst case. The filters reduce comparisons when candidates can be excluded early, but do not remove this dependence. Vectorised conflict counts can terminate once the allowed total is exceeded. A completed zero-conflict scan is reused while collecting missing loci; nonzero conflicts retain the original position-dependent error and missingness checks.

Population segments are assigned dynamically to OpenMP workers, retaining animal order within each segment. The pedigree team stays active across animals, with barriers between segment work and dependent orientation, crossover and parent updates. Parallel output reconstruction uses a nominal 32 MiB text/metadata budget and a 1,000-animal cap, allowing one oversized record; per-worker scratch is additional. A single writer preserves record order. Both engines are compiled from shared inference source and selected before allocation, without a mode branch inside the matching loops.

Source map

Mechanism	Tagged source file(s)
Raw genotype packing	genotype_cache.hpp
Packed working genotypes	genotype_workspace.hpp
Sampled-conflict rejection	candidate_index.hpp
Ordered lookup and frequency minima	high_memory_index.hpp
Matching, parallelism and output	findhap.cpp
LM cache and working buffers	external_genotype_cache.hpp, bounded_genotype_workspace.hpp
Mode selection	launcher.cpp

The archive includes the four manuscript figures and two performance heatmaps in PNG, SVG and PDF. Figure 1 shows a worked genotype-reconstruction example and the population workflow. Figure 2 compares candidate checking with and without the sampled-position filter. Figure 3 shows compact genotype storage, the two storage modes and worker reuse during pedigree processing. These diagrams illustrate the computation; they do not measure how much each optimisation contributes to speedup. The end-to-end benchmarks measure the combined implementation.

S4. Individual observations and calculated summaries

Tables S1 and S2 give all 42 observations; Table S3 gives ratios of medians. Table S4 summarises final-step positive-frequency haplotype counts extracted from the retained console logs; all six logs per dataset agree. In the reference code, these counts are calculated by counting entries with positive `hapfreq` after final-step output and assigning the count to `nhap(seg)`. They describe the final libraries, not peak occupancy, the largest identifier allocated, or candidate-search depth. Full chromosome/segment counts and log hashes are supplied in `data/turbohap_haplotype_libraries_221026.json`; `tools/summarise_haplotype_libraries.py` extracts the same table from a fresh run's log. F denotes Fortran, T standard TurboHap and LM TurboHap LM. Dimensions are genotyped animals × exported markers; k = 1,000 and M = 1,000,000. Values are rounded for display; the CSV and JSON retain full precision.

Table S1. Individual complete-process runtimes (seconds)

Animals × loci	F run 1	F run 2	T run 1	T run 2	LM run 1	LM run 2
10k × 1k	10.731	10.308	1.058	1.057	1.484	1.486
10k × 10k	74.956	76.045	2.637	2.644	2.934	3.048
10k × 100k	450.142	446.526	15.880	16.098	24.236	24.398
100k × 1k	871.573	876.206	15.192	14.884	21.076	21.577
100k × 10k	1187.166	1177.623	23.815	23.668	32.171	32.121
100k × 100k	4596.083	4644.075	153.392	154.708	237.038	237.588
1M × 10k	24715.297	24924.011	267.352	263.837	441.202	442.019

Table S2. Individual peak resident memory (GiB)

Animals × loci	F run 1	F run 2	T run 1	T run 2	LM run 1	LM run 2
10k × 1k	0.302	0.302	0.183	0.182	0.180	0.180
10k × 10k	1.197	1.197	0.766	0.765	0.746	0.746
10k × 100k	10.645	10.645	6.312	6.312	5.910	5.910
100k × 1k	0.377	0.377	0.331	0.332	0.316	0.316
100k × 10k	1.615	1.615	1.499	1.499	1.101	1.101
100k × 100k	14.985	14.985	13.025	13.026	8.581	8.581
1M × 10k	16.117	16.116	16.472	16.472	12.077	12.077

Table S3. Ratios of medians relative to Fortran

Animals × loci	T speedup	T RSS change	LM speedup	LM RSS change
10k × 1k	10.0×	-39.6%	7.1×	-40.5%
10k × 10k	28.6×	-36.1%	25.2×	-37.7%
10k × 100k	28.0×	-40.7%	18.4×	-44.5%
100k × 1k	58.1×	-12.2%	41.0×	-16.3%
100k × 10k	49.8×	-7.2%	36.8×	-31.8%
100k × 100k	30.0×	-13.1%	19.5×	-42.7%
1M × 10k	93.4×	+2.2%	56.2×	-25.1%

Table S4. Final-step positive-frequency haplotypes per segment

Animals × loci	Segments	Minimum	Median	Maximum
10k × 1k	30	602	3,035.0	5,128
10k × 10k	215	313	580.0	1,060
10k × 100k	2,016	107	165.0	322
100k × 1k	30	2,121	13,840.0	24,190
100k × 10k	215	1,050	2,306.0	4,838
100k × 100k	2,016	361	528.5	1,623
1M × 10k	215	3,226	9,993.0	19,861

S5. Synthetic input generation and identity

The following recipe produces the benchmark workloads. The dataset manifest records simulator source hashes, settings, chip membership, chromosome lengths, input and truth checksums, and validation counts.

22 October 2026. This recipe describes the seven synthetic datasets used in the manuscript. The accompanying manifest, `data/turbohap_benchmark_datasets_221026.json` in Supplementary File 2, records source hashes, dimensions, chip counts, chromosome lengths, the exact four benchmark inputs and retained genotype/haplotype truth identities.

Dataset sizes

The animal count in the tables is the number of **genotyped animals**. Each pedigree contains four additional ungenotyped founder females. The marker count is the number exported in `chromosome.data`; the simulator also creates 100 quantitative trait loci (QTLs), which are excluded from that marker map.

Genotyped animals	Pedigree animals	Exported markers	Simulated loci including QTLs	FindHap `maxhap`
10,000	10,004	1,000	1,100	50,000
10,000	10,004	10,000	10,100	50,000
10,000	10,004	100,000	100,100	50,000
100,000	100,004	1,000	1,100	50,000
100,000	100,004	10,000	10,100	50,000
100,000	100,004	100,000	100,100	50,000
1,000,000	1,000,004	10,000	10,100	500,000

Each dataset is simulated independently. Reusing the same seed does not make the smaller datasets subsets of the larger ones. The simulation uses eight initial ancestors and a simple pedigree with large families; the million-animal case has a parent with 499,998 progeny. These are size-varying benchmark workloads under a simple pedigree model.

There are 30 chromosomes with simulated lengths; chromosome 30 has X-specific handling. About 83.33% of animals have the full marker panel and 16.67% have a panel containing every third marker. Marker seed 96, initial linkage parameter 0.20, missing-call probability 0.01, erroneous-call probability 0.0002 and mutation probability 0.00000003 are used throughout.

The linkage parameter is applied between adjacent simulated loci. Keeping it at 0.20 while changing marker density does not preserve linkage disequilibrium over a fixed physical distance. Pedigree structure and haplotype diversity also change across these independently generated datasets. The size comparisons therefore describe the workloads tested, rather than isolating dimensions alone.

Build the simulator

Download and extract version 4 from the USDA GENOSIM page. The benchmark generation used GCC/gfortran 13.3.0 on Linux ARM64. Compare the source and option-file checksums with `simulator_sources` in the manifest; the version name alone does not identify an exact source snapshot.

In the directory containing the simulator sources:

```
gfortran kind.f90 pedsim.f90 -o pedsim
gfortran kind.f90 markersim.f90 -o markersim -ffp-contract=off
gfortran kind.f90 genosim.f90 -o genosim -O3 -ffp-contract=off
```

Both simulators reconstruct marker locations. On the tested ARM64 build, disabling floating-point contraction aligned their collision handling and chromosome assignments for all three panel sizes. Use the flags above to keep genotypes and truth aligned with the exported marker map.

These tools are needed only to generate benchmark data. They are not needed to build or run TurboHap. The following example generates 10,000 animals × 1,000 markers. Replace `/path/to/genosimV4` with your simulator directory.

```
mkdir simulated-10000-1000
cd simulated-10000-1000
cp /path/to/genosimV4/pedsim.options /path/to/genosimV4/markersim.options \
   /path/to/genosimV4/genosim.options /path/to/genosimV4/chips.txt .
```

Use a fresh directory for each dataset. The original simulation also writes large truth and intermediate files, so allow disk space beyond the four final FindHap inputs.

Set the dimensions and FindHap options

Run this block in the simulation directory. Change `animals` and `loci` to select another row above; retain the other simulator settings from the identified option files.

```
python3 - <<'PY'
from pathlib import Path

animals, loci = 10000, 1000
pedigree = animals + 4

ped = Path('pedsim.options').read_text().splitlines(True)
ped[3] = f'{pedigree} {pedigree // 3} {pedigree // 3} 2 2 10\n'
Path('pedsim.options').write_text(''.join(ped))

markers = Path('markersim.options').read_text().splitlines(True)
markers[0] = f'{loci + 100} 100 1.12 0 10\n'
Path('markersim.options').write_text(''.join(markers))

maxhap = 500000 if animals == 1000000 else 50000
Path('findhap.options').write_text(
    f'4 30 100 50 3 {maxhap} 1 1 1 1\n'
    'iters XChrom maxlen minlen steps maxhap hapout genout damout listout\n\n.004\n')
PY
```

All three benchmark programs use this same `findhap.options` file, with all output selectors set to 1 and no prior-haplotype files. The million-animal case needs the larger haplotype capacity; it therefore changes both the animal count and this capacity relative to the smaller cases.

Generate and identify the inputs

Run the three original programs in order, stopping if any fails:

```
set -e
LC_ALL=C /path/to/genosimV4/pedsim > output.pedsim 2>&1
python3 /path/to/supplement/tools/prepare_simulated_parent_sexes.py .
LC_ALL=C /path/to/genosimV4/markersim > output.markersim 2>&1
LC_ALL=C /path/to/genosimV4/genosim > output.genosim 2>&1
mv genotypes.raw genotypes.txt
```

The included sex-preparation script runs after `pedsim` and before `genosim`. For each parent, it sets sex to male for a sire role or female for a dam role, rejects parents used in both roles, and updates the associated pedsim metadata. It changes sex labels only, retaining ancestry and animal identifiers, and refuses to run if genotypes or truth already exist. Genotypes and truth must be generated after this preparation. The manifest records the changed-label counts for each dataset.

Keep `genotypes.true` and `haplotype.data`, optionally gzip-compressed, as the simulation truth. The audited truth includes all pedigree animals; inferred benchmark outputs must be kept separate from these simulator truth files. Before timing, check exported marker locations against the simulator replay, marker coverage, truth-phase/genotype consistency, parent roles and male X-chromosome haploidy. The completed checks and truth identities are in the manifest. Truth validation does not establish imputation accuracy.

Check that each log ends with its normal-completion message. `geno2seq` is unnecessary because these tests use SNP-chip genotypes, not sequence reads; no conflict-correction or phenotype-simulation step is applied to the inputs.

On Linux, obtain the four input fingerprints with:

```
sha256sum pedigree.file chromosome.data genotypes.txt findhap.options
```

Compare them with `inputs` for the selected dataset in the manifest. The checksums describe the actual benchmark inputs, including the larger `maxhap` options file for 1,000,000 × 10,000. Random-number implementations and floating-point behaviour can vary by compiler and platform: matching settings alone do not establish byte-identical regenerated inputs.

For timing, copy the four inputs into a separate clean run directory for each program and repetition. Use ten threads, `LC_ALL=C` and `OMP_DYNAMIC=FALSE`, and measure the complete executable, including all output writes. Compare outputs after timing, following the compatibility policy. Section S1 gives the run order, measurement conditions and compiler details; Section S2 gives the rerun command. The exact input datasets and simulation truth are available from Zenodo (doi:10.5281/zenodo.23566926) under CC BY 4.0, with per-dataset ZIP archives, documentation and checksums.

S6. Checking the supplementary archive

After extracting Supplementary File 2, run from its top-level directory:

```
python3 tools/recalculate_benchmarks.py
```

Python 3.9+ and its standard library are sufficient. This checks payload hashes, CSV/JSON agreement and the expected 42 records, then prints recalculated summaries. To repeat inference and output comparisons, use `tools/benchmark_dataset.py` with the input data and built executables. The figure scripts require NumPy and Matplotlib as documented in the archive README.

S7. Data availability and licences

All seven synthetic input datasets and retained genotype and haplotype truth are available in *TurboHap synthetic benchmark datasets and simulation truth*, version 1.0.0, on Zenodo under CC BY 4.0. The DOI is 10.5281/zenodo.23566926. The article and figures use CC BY 4.0. TurboHap retains its custom software licence, and the original Fortran reference retains its public-domain status. The supplementary archive supplies compact records, sources and tools; the full input data are in the Zenodo collection, and executable binaries are separate.